Graph Structure Is Necessary for Information Preservation Under Bounded Context

Patrick D. McCarthy

Abstract

Any system that (i) accumulates propositions about a world it cannot fully observe, (ii) operates under a bounded active context — a hard limit on the number of propositions simultaneously available for inference — and (iii) must preserve information as the proposition count grows beyond that bound, necessarily maintains a directed graph with typed edges over its propositions. We prove this claim from two independent arguments. The *information-preservation argument* shows that the only space-creating operation that does not lose information is factoring — extracting shared structure into named nodes with typed edges — and that factoring *is* graph construction. The *retention-dynamics argument* shows that any representation supporting sub-linear dependency queries, contextual retrieval, and selective removal with cascade necessarily encodes directed adjacency, which is a semantic graph regardless of substrate. We show that continuous-weight mechanisms (e.g., transformer self-attention) implement graph structure rather than constitute an alternative to it. The result is substrate-independent: it applies to biological neural networks, artificial neural networks, institutional knowledge systems, or any other medium that stores propositions under bounded context.

1 Introduction

Systems that accumulate knowledge face a fundamental tension. The knowledge base grows, but the capacity to reason over it at any single moment does not grow proportionally. Biological working memory holds roughly 7 ± 2 items [19]. Transformer context windows, though large, degrade in inference quality as they fill with content irrelevant to the current problem [16]. Institutional decision-makers cannot hold an entire organization's knowledge in a single meeting. In each case, there is a hard or effective bound on *active context*: the number of propositions simultaneously available for inference.

This paper asks a narrow question: **what structure must a knowledge representation have if information is not to be lost as the number of stored propositions exceeds the active context bound?**

The answer is a directed graph with typed edges. We prove this from two independent arguments, each sufficient on its own:

1. **Information preservation** (Theorem 1). When a bounded system must make space for new propositions, the only operation that creates space without losing information is *factoring*: extracting shared structure into a named node with typed edges. Factoring is graph construction. Discarding without adjacency information is blind; lossy compression without adjacency is blind discarding with extra steps.

2. **Retention dynamics** (Theorem 2). Any representation that supports sub-linear dependency queries — answering "what depends on this proposition?" in less than $O(n)$ time — necessarily stores directed adjacency. Selective removal with cascade (invalidating a proposition and propagating to its dependents) requires typed edges to scope correctly. These are the operations any system performs when maintaining knowledge under continuous evidence arrival.

The result is substrate-independent. It does not prescribe an implementation (adjacency lists, edge tables, attention masks, synaptic connections) — it proves that the *information content* of any adequate representation is a directed graph, regardless of how that graph is physically realized.

1.1 Related work

Graph-structured knowledge representations are widely used in practice: knowledge graphs [12], Bayesian networks [20], description logics [2], and the RDF/OWL stack [3] all represent knowledge as directed labeled graphs. Guarino's formal ontology [11] grounds this choice in the structure of the domain: entities, relations, and constraints map naturally to nodes, edges, and types. These traditions have demonstrated that graph structure is *effective* for knowledge representation. The question of whether it is *necessary* — as opposed to convenient — has received less formal attention.

The present paper provides the missing argument: under bounded active context, graph structure is not a design preference but a mathematical consequence of information preservation (Theorem 1) and sub-linear maintenance (Theorem 2). The typed directed edges of description logics and OWL are not an engineering convenience but the minimal structure any system must maintain. This result retroactively grounds decades of practice in a formal necessity argument.

Dupoux, LeCun, and Malik [9] propose a dual-system architecture (observation + action) with meta-control routing, motivated by cognitive science. Their framework is architectural and prescriptive: it argues that autonomous learning *should* integrate observation, action, and meta-control. The present work is complementary: rather than proposing what intelligent systems should contain, we derive what they *must* contain under bounded context.

The information bottleneck principle [24] establishes that optimal representations compress input while preserving information relevant to output. Our information-preservation argument (Theorem 1) addresses a different question: not what to compress, but what structure the representation must have to compress without information loss.

Simon's bounded rationality [22] establishes that cognitive agents satisfice rather than optimize. Our bounded context constraint (C_n) formalizes a specific consequence: when the bound is hard, structure is not optional but necessary.

2 Formal Framework

We introduce the minimal definitions required for the two main results.

Definition 1 (World State Space)**.** Let W be a measurable space of possible world states. At time t, the true state $w_t \in W$ is not directly observable by any entity.

Definition 2 (Proposition and Knowledge Base)**.** A *proposition* p is a claim about W with an associated likelihood function $\ell_p : W \to [0, 1]$. A *knowledge base* K is a collection of propositions.

The posterior over world states given K is:

$$P(w \mid K) \propto P_0(w) \cdot \prod_{p \in K} \ell_p(w)$$

where P_0 is a prior. The *uncertainty* of K about w is the surprisal $U(w, K) = -\log P(w \mid K)$ [21, 5].

Definition 3 (Bounded Active Context)**.** For an entity n, let C_n denote its *active context capacity*: the maximum number of propositions simultaneously available for inference. Let $\mathrm{ctx}(t) \subseteq K$ be the set of propositions loaded into active context at time t, with $|\mathrm{ctx}(t)| \leq C_n$.

The *inference quality* for problem f is the relevance density:

$$\rho(f) = \frac{|K^{[f]}|}{|\mathrm{ctx}(t)|}$$

where $K^{[f]} \subseteq K$ is the set of propositions relevant to f. $\rho = 1$ when active context holds exactly the relevant propositions; $\rho \to 0$ when context is dominated by irrelevant content.

Definition 4 (Growth and Curation Regimes)**.** K has a maximum capacity K_{max} determined by substrate constraints. When $|K| < K_{max}$, K is in the *growth regime*: propositions accumulate freely. When $|K| = K_{max}$, K is in the *curation regime*: adding a new proposition requires making space.

Definition 5 (Contextual Retrieval)**.** Given problem f, *contextual retrieval* is the operation that loads $K^{[f]}$ into active context. The retrieval cost is the number of propositions examined to identify $K^{[f]}$.

3 Graph Necessity from Information Preservation

Definition 6 (Factoring)**.** Let $p_1, \ldots, p_m \in K$ share mutual information: there exists a random variable F^* (a function of W) such that $I(F^*; \ell_{p_i}) > 0$ for all i, and the joint likelihood decomposes as

$$\ell_{p_i}(w) = g_i\big(\ell_{p_i}^{\mathrm{res}}(w),\ F^*(w)\big)$$

for some residual $\ell_{p_i}^{\mathrm{res}}$ and reconstruction function g_i. The decomposition need not be unique; any valid decomposition suffices. In the general case, g_i may be an arbitrary measurable function — the definition does not require multiplicative, additive, or any other restricted functional form. *Factoring* is the operation that:

(i) Extracts F^* into a single named, reusable unit.

(ii) Replaces the shared component in each p_i with a reference to F^*.

(iii) Stores, for each reference, the reconstruction function g_i specifying the role F^* plays in recovering p_i.

The total information is preserved: F^* plus the residuals and their reconstruction functions jointly recover all original ℓ_{p_i}.

Note: this definition is stated without graph vocabulary. The output of factoring is a set of units with references between them and role annotations on each reference. We prove in Theorem 1 that this structure necessarily constitutes a directed graph with typed edges.

Remark. Factoring is not a novel operation introduced by this paper. It is the central operation of description logics [2]: the TBox/ABox separation extracts shared conceptual structure (the TBox) from individual assertions (the ABox), and concept subsumption hierarchies are literally shared components referenced by multiple individuals. Upper ontologies such as BFO [1] and DOLCE [15] carry this further: approximately 35 categories in BFO organize all of biomedical knowledge, demonstrating that real-world knowledge bases have abundant shared structure amenable to factoring. The definition above formalizes what these traditions have practiced for decades.

Theorem 1 (Graph Necessity from Information Preservation)**.** *Let K be a knowledge base with bounded active context C_n (Definition 3) in the curation regime (Definition 4). Any representation of K that preserves information as $|K|$ grows beyond C_n necessarily maintains a directed graph with typed edges over propositions.*

Proof. In the curation regime, adding any new proposition p_{new} requires making space. We enumerate the ways to make space and show that every information-preserving option produces graph structure.

Option A: Discard an existing proposition. Without explicit relational structure, the entity has no mechanism to determine which existing propositions are redundant. Two propositions p_1, p_2 that share a latent factor appear as independent entries; there is no stored evidence that they share structure. The choice of which to discard is therefore made without information about relational dependencies. Discarding p_{old} loses the information it encodes, and potentially loses the information that p_{old} has any relation to other propositions — a second-order loss. This is information loss.

Option B: Factor — extract shared structure (Definition 6). If propositions share a common informational component F^*, factoring extracts F^* as a single named node with typed edges, replacing redundant copies with shared references. Space is created without loss: the original information is reconstructable from F^* plus the residuals. Factoring also resolves the update problem: when new evidence revises F^*, the single node is updated once, and the revision propagates to all propositions that reference it via edges — $O(1)$ instead of $O(|K|)$.

By Definition 6, factoring produces nodes and directed typed edges. This is graph construction.

Option C: Reorganization — rewrite propositions into a more compact equivalent encoding. Propositions $p_1, \ldots, p_m$ could be replaced by $q_1, \ldots, q_k$ ($k < m$) that are jointly equivalent: the same posterior over W is induced. This is a change of basis in the information-theoretic sense.

If the reorganization preserves all dependency relationships — which propositions ground which — then the result is an isomorphic knowledge base with fewer nodes. But "preserving dependency relationships" requires storing those relationships, which is storing adjacency. If the reorganization does *not* preserve dependency relationships, then downstream propositions that depended on the original p_i are silently broken: they reference nodes that no longer exist, with no forwarding information. The entity cannot determine what broke without adjacency.

There is a special case: reorganization that extracts shared structure across the p_i into shared nodes referenced by the q_j. This is factoring (Option B). Reorganization that does not extract shared structure but merely rewrites individual propositions more compactly does not create space from shared structure — it creates space from per-proposition compression, which is lossless compression (Option D).

Option D: Lossless compression — encode propositions more compactly without information loss. Lossless compression reduces the physical footprint of a knowledge store but does not change the hard capacity bound C_n. The bound is a constraint on how many propositions can be *simultaneously active for inference*, not on how compactly they are encoded on disk. A compressed bitstring must be decompressed before any proposition can be accessed or any dependency traced; the decompressed form faces the same curation pressure as the original. Compression is orthogonal to the structural question the theorem addresses.

Option E: Lossy compression — replace propositions with a summary that occupies less space. Compressing p_1 and p_2 into a summary p_s that discards detail is a form of discarding: the lost detail may ground other propositions or actions. If p_1 has dependents — propositions that reference it — then replacing p_1 with p_s silently invalidates those dependents unless the entity knows what depends on the lost detail. But knowing what depends on p_1 *is* knowing p_1's adjacency — its edges. Without stored adjacency, lossy compression is blind discarding: the entity cannot evaluate what breaks. With stored adjacency, the entity can scope the damage, notify dependents, and compress safely — but storing adjacency is maintaining a graph. Lossy compression that preserves correctness requires the same relational structure as factoring. Lossy compression that ignores relational structure is Option A with extra steps.

Exhaustiveness. Options A–E cover the space of space-creating operations: remove information (A, E), extract shared structure (B), rewrite to equivalent form (C), or encode more compactly (D). Of these, A and E lose information. D is orthogonal (it addresses storage density, not the active-context bound). C either reduces to B (when shared structure is extracted) or requires stored adjacency to preserve dependencies. Only B (factoring) creates space, preserves information, and solves the structural problem.

The output of factoring is a directed graph with typed edges. Definition 6 is stated without graph vocabulary: it produces named units with references and reconstruction roles. We now show this structure has graph properties:

Nodes. The named units — shared components F^* and residual propositions p_i^{res} — are the nodes. Each is an identifiable, individually addressable unit of information.

Directed edges. Each reference from p_i^{res} to F^* is directed: p_i depends on F^* for reconstruction, but F^* does not depend on p_i. Revising F^* requires updating the reconstruction of every p_i that references it; revising p_i does not require updating F^*. This asymmetry is inherent in the extraction operation: the shared component is referenced by the residuals, not vice versa. Any structure encoding asymmetric dependencies between named units is a directed graph.

Typed edges. The reconstruction role g_i stored on each reference distinguishes the *kind* of dependency: whether F^* *grounds* p_i (the residual's truth value depends on F^*), *evidences* p_i (the residual's likelihood is informed by F^*), or *causes* p_i (the residual's state is produced by F^*). Without role annotation, cascade from an invalidated F^* cannot distinguish dependents that require re-evaluation (inferentially grounded through F^*) from units merely associated with F^*. Untyped references force worst-case cascade over all references; typed references scope cascade to dependency-bearing references only. Since the reconstruction role g_i is stored by Definition 6(iii), the edges are typed.

A collection of named units with directed, typed references between them is a directed graph with typed edges, regardless of whether it is implemented as adjacency lists, pointers, sparse matrices, or any other medium.

Remark on shared structure. The theorem requires that propositions share informational components. This is not an additional assumption but a consequence of the knowledge base describing

a structured world. The proposition "fire is hot" is relevant to cooking, camping, metallurgy, and fire safety — any knowledge base spanning these domains contains this shared component. A knowledge base with *zero* shared structure would be a collection of completely independent facts about completely unrelated domains: each proposition grounded in its own private ontology with no overlap. Such a knowledge base faces no curation pressure (nothing is redundant, so nothing can be factored) and requires no structural optimization. The theorem addresses the non-degenerate case: knowledge bases about structured worlds, which is every knowledge base that admits inference across domains. The success of upper ontologies confirms that this is the typical case: BFO [1] organizes all of biomedical science under approximately 35 shared categories; DOLCE [15] provides cross-domain foundational categories reused across engineering, geography, and law. These are not design choices but reflections of shared structure that already exists in the domains they describe.

Remark on compression. Lossless compression changes how compactly propositions are stored, not how many can be simultaneously active. The bounded-context premise ($|K| > C_n$) is a constraint on active inference capacity, not storage density. No encoding scheme removes this bound; compression is orthogonal to the structural question. □

4 Graph Necessity from Retention Dynamics

Theorem 2 (Graph Necessity from Retention Dynamics)**.** *Let K be a knowledge base supporting continuous knowledge maintenance under evidence arrival. The maintenance operations are:*

(i) ***Addition with provenance****: insert a proposition p together with its evidential grounds — which existing propositions p depends on, and by what relation.*

(ii) ***Contextual retrieval****: given problem f, retrieve exactly $K^{[f]}$.*

(iii) ***Dependency query****: given proposition p, determine which propositions are grounded through p.*

(iv) ***Selective removal with cascade****: when p is invalidated, remove p and re-evaluate every proposition whose grounding depends on p, transitively, until K stabilizes.*

Any representation achieving sub-linear cost on all four operations simultaneously necessarily encodes directed adjacency between propositions — and any structure encoding directed adjacency over propositions with typed relations is *a semantic graph, regardless of implementation.*

Proof. **Part 1: Dependency queries are necessary and continuous.** Knowledge maintenance operates continuously: every new piece of evidence potentially shifts the status of existing propositions. When evidence revises or invalidates a proposition p, the system must evaluate every proposition whose grounding passes through p. This is not a batch operation that can be deferred — acting on a proposition whose ground has been invalidated risks incorrect inference. The dependency query "what is grounded through p?" is therefore invoked at the frequency of evidence arrival.

Part 2: Lower bound without adjacency. Consider a representation R that stores n propositions without explicit directed relationships between them (a flat store, hash table, embedding space, or any structure that does not record which propositions depend on which). To answer the dependency query for p, R must examine each of the other $n - 1$ propositions to determine

whether its grounding involves p. Without stored adjacency, there is no mechanism to restrict this examination: any proposition might depend on p, and the only way to determine this is to check.

The cost of a single dependency query is therefore $\Omega(n)$ for any representation without stored adjacency. Since dependency queries are invoked at evidence-arrival frequency (Part 1), the amortized cost of maintenance is $\Omega(n)$ per evidence event — growing linearly with $|K|$.

Part 3: Cascade amplifies the cost. Selective removal is not a single deletion. When p is removed, each proposition q grounded through p must be re-evaluated: does q have alternative grounds, or should it also be removed? If q also fails, its dependents must be re-evaluated in turn. This is a transitive closure on the dependency relation.

With explicit adjacency, cascade follows edges: cost is $O(|D_p|)$ where $|D_p|$ is the transitive dependency set. Without adjacency, each level of cascade requires an $\Omega(n)$ scan. Total cost: $\Omega(n \cdot L)$ where L is the cascade depth, versus $O(|D_p|)$ in the graph where $|D_p| \ll n$ for typical propositions with bounded connectivity.

Remark. Truth maintenance systems [8, 6] are a direct implementation of operations (iii) and (iv). A TMS stores directed justifications (which propositions ground which) and propagates belief revision through them when any justification is invalidated — precisely the dependency query and cascade operations this theorem proves are necessary. The fact that AI research independently invented TMS to solve this exact problem is empirical evidence for the theorem's claim: without stored directed adjacency, maintenance under evidence arrival is intractable.

Part 4: Any sub-linear solution encodes adjacency. Suppose R achieves $o(n)$ cost for dependency queries. Then R must store, for each proposition, information about which other propositions depend on it — otherwise the $\Omega(n)$ lower bound of Part 2 applies. This stored information is a set of directed relationships: for each proposition p, a record of the propositions that reference p as part of their grounding.

This is precisely a directed adjacency structure. The nodes are propositions. The edges are directed dependency relations. Any representation that stores this information — regardless of whether it is implemented as adjacency lists, edge tables, pointers, or any other mechanism — *is* a directed graph over propositions.

Remark on approximate methods. Approximate retrieval mechanisms — learned embeddings, locality-sensitive hashing, approximate nearest-neighbor indices — achieve sub-linear performance on operation (ii) (contextual retrieval) without storing explicit directed adjacency. This does not circumvent the theorem, because the theorem requires sub-linear cost on all four operations *simultaneously*. Approximate similarity over an embedding space can identify propositions *related to* a query, but cannot answer "what is *grounded through* p?" (operation iii) or propagate invalidation transitively through dependency chains (operation iv). These operations require knowing which propositions depend on which — not which are *nearby* in a metric space. Proximity and dependency are distinct relations: p and q may be embedding-close yet inferentially independent, or embedding-distant yet linked by a grounding chain. Any representation that achieves sub-linear cost on dependency queries and cascade necessarily stores directed dependency relationships, which is directed adjacency.

Part 5: Typed edges are necessary for correct cascade. A bare adjacency structure (untyped edges) is insufficient. When p is invalidated, cascade must distinguish between propositions *inferentially grounded* through p (which require re-evaluation) and propositions merely *associated* with p (which do not). An untyped edge conflates these: every neighbor must be treated as potentially dependent, degrading cascade to the full neighborhood rather than the true dependency set.

Typed edges — carrying the relation type ("grounds," "evidences," "causes," "supports") — enable cascade to follow only dependency-bearing edges, keeping cost proportional to $|D_p|$ rather than $|\text{adj}(p)|$. Since $|\text{adj}(p)|$ can be much larger than $|D_p|$, typed edges are necessary for efficient cascade.

Remark on empirical convergence. The necessity of typed edges is confirmed by the empirical history of knowledge representation. Early semantic networks used untyped "is-related-to" links; Woods [25] demonstrated that these were semantically ambiguous and operationally inadequate — the system could not distinguish inheritance from association, causation from correlation. The field converged to typed relations: RDF introduced typed predicates [3], OWL distinguished object properties, data properties, and annotation properties with transitive, symmetric, and inverse role characteristics [2]. This trajectory — starting untyped, failing, converging to typed — is exactly the progression Part 5 predicts must occur.

A directed graph with typed edges over propositions is a semantic graph. □

5 Continuous-Weight Mechanisms Are Graph Implementations

A natural objection: modern attention mechanisms achieve contextual retrieval without explicit graph construction. Does this constitute a non-graph alternative?

Theorem 3 (Contextual Retrieval Requires Adjacency Computation)**.** *Any mechanism that performs contextual retrieval over n propositions — selecting which propositions are relevant to a given query — must compute pairwise relevance between the query and stored propositions. This computation either (a) evaluates all n propositions per query at $O(n)$ or $O(n^2)$ cost (dynamic adjacency), or (b) pre-computes and stores relevance relationships at $O(d)$ per query where d is the local degree (stored adjacency). In case (b), the stored relationships constitute a directed graph. Case (a) is the cost of recomputing that graph from scratch at every query.*

Proof. Contextual retrieval for problem f requires identifying $K^{[f]}$ — the propositions relevant to f. This requires determining, for each proposition $p \in K$, whether p is relevant to f. Without stored information about relevance relationships, every proposition must be evaluated: cost $\Omega(n)$ per query.

Full self-attention is the canonical example: $A_{ij} = \text{softmax}(Q_i K_j^\top / \sqrt{d_k})$ evaluates all n^2 pairwise relevances per query. This is not "computing a graph" in the trivial sense that any matrix can be called a graph — it is performing the specific computation (pairwise relevance determination) that a pre-built graph would make unnecessary. The attention matrix is dense: all n^2 entries are nonzero, encoding no structural information. The computational cost is the cost of having no stored structure.

Pre-computed adjacency — stored information about which propositions are relevant to which queries — reduces retrieval to $O(d)$ where d is the number of stored relevant relationships. This is a directed graph: nodes are propositions, edges are relevance relationships, and the direction encodes which proposition informs which query context.

Sparse attention mechanisms — learned sparsity patterns, local windows, routing functions — reduce the $O(n^2)$ cost by learning which pairwise relevances are worth computing: they are learning which edges to store. The KV cache materializes computed relevances for reuse across queries. Each optimization converts dynamic adjacency computation into stored adjacency — progressively building the graph that Theorems 1 and 2 prove is necessary.

The result is not that "attention is a graph" (which would be trivially true of any matrix) but that contextual retrieval requires adjacency information, and the only question is whether that information is computed dynamically ($O(n^2)$) or stored explicitly ($O(d)$). Stored adjacency over propositions with typed relevance relationships is a directed graph. □

Remark (Scaling context windows). Growing C_n directly does not solve the retrieval problem. For any specific problem f, the relevance density $\rho(f) = |K^{[f]}|/|\mathrm{ctx}(t)|$ (Definition 3) decreases as context fills with content not relevant to f — an effect documented empirically as "lost in the middle" [16]. The efficient path is not to grow the context window but to grow the encoded knowledge accessible via stored adjacency: filling the graph, not the context window.

6 Discussion

The two theorems establish graph necessity from independent premises. Theorem 1 requires only the curation regime (bounded capacity, new propositions arriving). Theorem 2 requires only continuous knowledge maintenance (evidence arrives, dependencies must be traced) and does not assume lossless preservation — it applies equally to systems that tolerate information loss, because the cost argument concerns maintenance operations, not retention policy. Any system subject to both constraints — which includes every physical system that accumulates knowledge — is doubly committed to graph structure.

6.1 The continuous-representation objection

The strongest objection to this paper's results is that they assume discrete propositions, while the most capable knowledge systems — large language models, biological neural networks — store knowledge in continuous distributed representations that do not decompose into identifiable propositions. If there are no discrete propositions, there is nothing to factor (Theorem 1), nothing to query dependencies over (Theorem 2), and calling an attention matrix a "graph" (Theorem 3) is a mathematical relabeling, not a structural discovery. The objection: the theorems prove graph necessity for an idealized system that no successful architecture actually implements.

This objection is serious and requires a substantive response. We offer three.

(1) Continuous representations do encode discrete propositions. The continuity of the substrate does not entail that the information content is non-propositional. Meng et al. [18] demonstrate that specific factual associations in GPT-class models can be localized, edited, and ablated at identified parameter subsets: "The Eiffel Tower is in Paris" has a locatable representation that can be surgically revised to "The Eiffel Tower is in Rome" without retraining. Geva et al. [10] show that feed-forward layers in transformers function as key-value memories, with individual neurons activating for identifiable semantic patterns. Li et al. [14] trace factual recall to discrete circuits within continuous networks.

These results establish that continuous weight matrices encode propositional content — not approximately or metaphorically, but in a sense precise enough to support surgical editing. The propositions are *superposed* (multiple propositions share parameters) and *distributed* (each proposition spans multiple parameters), but they are identifiable, editable, and individually removable. Superposed encoding is precisely the condition that makes factoring valuable: shared structure across propositions is the input to factoring, and continuous networks contain it abundantly.

(2) The claim is about information content, not implementation. Theorems 1 and 2 prove that the *information content* of any adequate representation has graph structure. They do not prescribe adjacency lists, edge tables, or symbolic labels. A neural network whose weights encode directed dependencies between propositions *is* maintaining a graph in the sense of Definition 2 — the graph exists in the information, not in the data structure.

The objection that "if any structured representation counts as a graph, the claim is trivially true" misidentifies the contribution. The claim is not that representations have structure (which is trivial) but that the structure must be *directed*, *typed*, and *adjacency-preserving* — and that representations lacking these specific properties lose information or pay $\Omega(n)$ maintenance costs. Not all structure is graph structure. A flat embedding space has structure (metric, topology) but not directed adjacency; a hash table has structure (key-value mapping) but not typed dependencies. The theorems identify *which* structural properties are necessary, not merely that some structure is required.

(3) Catastrophic forgetting is the theorem's prediction for systems without explicit adjacency. This is the strongest empirical evidence for the result. Theorem 2 proves that without stored directed adjacency, selective removal with cascade — updating one proposition and propagating correctly to its dependents — costs $\Omega(n)$ per evidence event. The prediction: systems that store propositions in continuous weights without explicit adjacency should exhibit *uncontrolled interference when knowledge is updated*.

This is precisely catastrophic forgetting [17, 13]: updating a neural network on new information degrades previously learned information unpredictably. The network cannot scope the update because it lacks the adjacency to determine what depends on the modified parameters. Continual learning remains an open problem in deep learning not because of insufficient engineering but because the underlying representation lacks the structural property the theorem identifies: directed adjacency that enables scoped updates.

The response to the continuous-representation objection is therefore not defensive but *predictive*: the theorem identifies the specific structural deficit (missing explicit adjacency), predicts the specific failure mode (unscoped updates causing cascading interference), and the most successful continuous-representation systems exhibit exactly that failure mode. The objection that the theorems do not apply to neural networks is refuted by the fact that neural networks suffer from precisely the pathology the theorems predict for representations without graph structure.

(4) Knowledge graph embeddings are continuous implementations of graph structure. A further data point: knowledge graph embedding methods (TransE [4], RotatE [23]) project graph structure into continuous vector spaces. The embeddings are continuous, but the information content they encode — which entities are related to which, by what relation — is directed typed adjacency. Inference tasks such as link prediction require reconstructing adjacency from the embedding space. These methods do not eliminate graph structure; they implement it in a continuous substrate, which is exactly the relationship Theorems 1–2 predict between information content (graph) and implementation (free parameter).

6.2 Scope and non-claims

The theorems prove that the *information content* of the representation must be a directed graph. They do not prescribe a specific graph formalism (property graph, RDF, Bayesian network) or implementation (adjacency lists, sparse matrices, attention masks). The graph is the abstract structure; the substrate is a free parameter.

Implications for AI architecture. Theorem 3 reframes the relationship between graph-based

and neural approaches: they are not competing paradigms but different implementations of the same structural necessity. Systems that compute adjacency dynamically (full attention) pay $O(n^2)$; systems that store it explicitly (graphs, sparse attention, KV caches) pay $O(d)$. The trend in architecture design — from dense attention toward sparse, structured, and cached retrieval — is convergence toward explicit graph structure, driven by the same cost pressures the theorems identify.

The result suggests that hybrid architectures combining explicit knowledge graphs with neural retrieval are not an engineering convenience but a response to a structural constraint: as $|K|$ grows, systems must encode adjacency to maintain inference quality under bounded context.

Implications for scaling. The scaling remark following Theorem 3 challenges the prevailing approach of growing context windows as the primary path to more capable inference. Under bounded context, larger windows with undifferentiated content *reduce* inference quality for any specific problem. The alternative — encoding knowledge into a graph and retrieving the relevant subgraph per problem — maintains $\rho = 1$ regardless of total knowledge size. This is the difference between scaling the window and scaling the knowledge.

Biological evidence. Biological neural networks maintain sparse, structured connectivity rather than dense all-to-all connections. Working memory is bounded ($C_n \approx 7 \pm 2$) while long-term knowledge grows without apparent limit [19]. Retrieval is associative and context-driven — consistent with graph traversal from a query node rather than exhaustive scan. The theorems suggest this architecture is not an accident of evolutionary history but a necessary consequence of bounded context under information-preservation pressure.

7 Conclusion

We have shown that graph structure over propositions is not a design choice but a necessary consequence of two independently sufficient constraints: information preservation under bounded capacity, and sub-linear knowledge maintenance under continuous evidence arrival. Continuous-weight mechanisms such as transformer self-attention implement graph structure dynamically rather than constituting alternatives to it. The result is substrate-independent: any system that accumulates propositions under bounded active context and must preserve information necessarily maintains a directed graph with typed edges, whether that graph is realized as explicit edges, learned sparse attention, synaptic connectivity, or any other medium.

References

[1] R. Arp, B. Smith, and A. D. Spear. *Building Ontologies with Basic Formal Ontology.* MIT Press, 2015.

[2] F. Baader, D. Calvanese, D. L. McGuinness, D. Nardi, and P. F. Patel-Schneider, editors. *The Description Logic Handbook: Theory, Implementation, and Applications.* Cambridge University Press, 2003.

[3] T. Berners-Lee, J. Hendler, and O. Lassila. The semantic web. *Scientific American*, 284(5):34–43, 2001.

[4] A. Bordes, N. Usunier, A. Garcia-Durán, J. Weston, and O. Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*, volume 26, pages 2787–2795, 2013.

[5] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

[6] J. de Kleer. An assumption-based TMS. *Artificial Intelligence*, 28(2):127–162, 1986.

[7] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 2021.

[8] J. Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979.

[9] E. Dupoux, Y. LeCun, and J. Malik. Why AI systems don't learn and what to do about it: Lessons on autonomous learning from cognitive science. *arXiv preprint arXiv:2603.15381*, 2026.

[10] M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5484–5495, 2021.

[11] N. Guarino. Formal ontology and information systems. In *Proceedings of FOIS'98*, pages 3–15. IOS Press, 1998.

[12] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier, A.-C. Ngonga Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.

[13] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[14] K. Li, O. Press, and S. Arora. How does GPT-2 compute greater-than?: Interpreting mathematical abilities in a pre-trained language model. In *Advances in Neural Information Processing Systems*, 2024.

[15] C. Masolo, S. Borgo, A. Gangemi, N. Guarino, and A. Oltramari. WonderWeb deliverable D18: Ontology library (final). Technical report, ISTC-CNR, 2003.

[16] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[17] M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In G. H. Bower, editor, *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

[18] K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, pages 17359–17372, 2022.

[19] G. A. Miller. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81–97, 1956.

[20] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

[21] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[22] H. A. Simon. A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1):99–118, 1955.

[23] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *Proceedings of the International Conference on Learning Representations*, 2019.

[24] N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. In *Proceedings of the 37th Annual Allerton Conference on Communication, Control, and Computing*, pages 368–377, 2000.

[25] W. A. Woods. What's in a link: Foundations for semantic networks. In D. G. Bobrow and A. Collins, editors, *Representation and Understanding: Studies in Cognitive Science*, pages 35–82. Academic Press, 1975.